

DGA Multi-class Classification — SVM (Linear)

Frames DGA detection as an **8-class problem** (7 DGA families + benign).

Section	Content
1–3	Imports, data loading, TF-IDF feature extraction
4–5	Train/test split + Linear SVM with C tuned via CV
6	CV grid search results
7	Per-class metrics table
8	Confusion matrix
9–11	t-SNE, UMAP, side-by-side comparison
12	Centroid distance matrix
13	Decision boundary (2-D PCA)
14	Feature importance — top n-grams per class (mean + contrast)
15	Summary

Features: character n-gram TF-IDF only (`char_wb` , 2–4-grams, top-5000).

Kernel: Linear — `C` tuned via 5-fold stratified CV (macro-F1).

1. Imports

```
In [1]: pip install -q scikit-learn umap-learn pandas numpy matplotlib seaborn scipy
```

Note: you may need to restart the kernel to use updated packages.

```
In [2]: import warnings, math, time
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from scipy.spatial.distance import cdist

from sklearn.svm import SVC, LinearSVC
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.model_selection import (
    train_test_split, StratifiedKFold, GridSearchCV
)
```

```

from sklearn.metrics import (
    classification_report, confusion_matrix, f1_score
)
import umap

SEED = 42
np.random.seed(SEED)
print('All imports OK')

```

All imports OK

2. Load Data & Define Labels

```

In [3]: dga_dataset = pd.read_csv('C:/Users/Admin/OneDrive - National University of Sing

# Benign rows get their own class label
dga_dataset = dga_dataset.dropna(subset=['domain']).copy()
dga_dataset['subclass'] = dga_dataset['subclass'].fillna('legit')
dga_dataset.loc[dga_dataset['isDGA'] != 'dga', 'subclass'] = 'legit'

CLASSES = sorted(dga_dataset['subclass'].unique().tolist())
N_CLASSES = len(CLASSES)

print(f'Classes ({N_CLASSES}): {CLASSES}')
print()
print('Class distribution:')
print(dga_dataset['subclass'].value_counts().to_string())

```

Classes (8): ['bamital', 'cryptolocker', 'gameoverdga', 'goz', 'legit', 'necurs', 'newgoz', 'nivdort']

Class distribution:

subclass	
legit	79998
cryptolocker	37254
newgoz	9276
gameoverdga	8461
nivdort	8456
necurs	8331
goz	6136
bamital	2086

3. Feature Extraction — Character N-gram TF-IDF

Each domain is treated as a character sequence. A TF-IDF vectoriser extracts weighted frequencies of **2-4 character shingles** using `char_wb` mode (pads word boundaries so short domains are handled cleanly). `sublinear_tf=True` log-compresses raw counts to reduce the effect of very frequent n-grams. The sparse matrix is kept for the SVM; a dense copy is used for embeddings and the centroid analysis.

```

In [4]: domains = dga_dataset['domain'].tolist()
labels = dga_dataset['subclass'].tolist()

print('Fitting TF-IDF on character n-grams (2-4)...')
tfidf = TfidfVectorizer(
    analyzer = 'char_wb',

```

```

    ngram_range = (2, 4),
    max_features = 5000,
    sublinear_tf = True,
)
X_sparse = tfidf.fit_transform(domains)
X_dense = X_sparse.toarray().astype(np.float32)

le = LabelEncoder()
y = le.fit_transform(labels)

PALETTE = plt.cm.get_cmap('tab10', N_CLASSES)
CLASS_COLORS = {i: PALETTE(i) for i in range(N_CLASSES)}

print(f'Feature matrix : {X_dense.shape[0]:,} x {X_dense.shape[1]:,}')
print(f'Classes : {list(le.classes_)}')

```

Fitting TF-IDF on character n-grams (2-4)...

Feature matrix : 159,998 x 5,000

Classes : [np.str_('bamital'), np.str_('cryptolocker'), np.str_('gameoverdga'), np.str_('goz'), np.str_('legit'), np.str_('necurs'), np.str_('newgoz'), np.str_('nivdort')]

4. Train / Test Split

```

In [5]: idx_all = np.arange(len(y))
        idx_tr, idx_te = train_test_split(idx_all, test_size=0.2,
                                         random_state=SEED, stratify=y)

X_train, X_test = X_sparse[idx_tr], X_sparse[idx_te]
X_train_d, X_test_d = X_dense[idx_tr], X_dense[idx_te]
y_train, y_test = y[idx_tr], y[idx_te]

print(f'Train : {X_train.shape[0]:,} | Test : {X_test.shape[0]:,}')
print()
print('Test set class counts:')
for cls_idx, cls_name in enumerate(le.classes_):
    print(f' {cls_name:15s}: {(y_test == cls_idx).sum():,}')

```

Train : 127,998 | Test : 32,000

Test set class counts:

bamital	: 417
cryptolocker	: 7,451
gameoverdga	: 1,692
goz	: 1,227
legit	: 16,000
necurs	: 1,666
newgoz	: 1,856
nivdort	: 1,691

5. Linear SVM Training — C Tuned via Cross-Validation

Kernel is fixed to **linear**. Grid search over:

- C — regularisation (0.001 → 10)

Scored by **macro-F1** so all 8 classes are weighted equally. `class_weight=balanced` compensates for imbalanced class sizes.

Set `N_CV_SAMPLES = None` to run CV on the full training set.

```
In [6]: N_CV_SAMPLES = 2000

if N_CV_SAMPLES and X_train.shape[0] > N_CV_SAMPLES:
    idx_sub = np.random.choice(X_train.shape[0], N_CV_SAMPLES, replace=False)
    X_cv, y_cv = X_train[idx_sub], y_train[idx_sub]
    print(f'CV sub-sample : {N_CV_SAMPLES:,} / {X_train.shape[0]:,}')
else:
    X_cv, y_cv = X_train, y_train
    print(f'CV on full train set : {X_train.shape[0]:,}')

# Linear SVM: only C needs tuning (no gamma)
param_grid = {
    'svm__C': [0.001, 0.01, 0.1, 1, 10],
}

pipe = Pipeline([
    ('scaler', StandardScaler(with_mean=False)),
    ('svm', LinearSVC(class_weight='balanced',
                      max_iter=2000,
                      random_state=SEED)),
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
gs = GridSearchCV(pipe, param_grid, cv=cv,
                  scoring='f1_macro', n_jobs=-1, verbose=1)
gs.fit(X_cv, y_cv)

best_C = gs.best_params_['svm__C']
print(f'\nBest C : {best_C}')
print(f'Best CV F1 : {gs.best_score_:.4f}')

# LinearSVC does not support predict_proba natively;
# wrap with CalibratedClassifierCV to enable probability estimates.
from sklearn.calibration import CalibratedClassifierCV

final_pipe = Pipeline([
    ('scaler', StandardScaler(with_mean=False)),
    ('svm', CalibratedClassifierCV(
        LinearSVC(C=best_C, class_weight='balanced',
                  max_iter=2000, random_state=SEED),
        cv=3
    ))
])
print('\nFitting final model on full training set...')
final_pipe.fit(X_train, y_train)

y_pred = final_pipe.predict(X_test)
y_proba = final_pipe.predict_proba(X_test)

# — Per-sample inference latency (main model) —————
BATCH_SIZE_LAT = 256
_sample_times = []
for _start in range(0, X_test.shape[0], BATCH_SIZE_LAT):
```

```

_batch = X_test[_start : _start + BATCH_SIZE_LAT]
_t0 = time.perf_counter()
final_pipe.predict(_batch)
_t1 = time.perf_counter()
_ms_per_sample = (_t1 - _t0) * 1000 / _batch.shape[0]
_sample_times.extend([_ms_per_sample] * _batch.shape[0])

_times_arr = np.array(_sample_times)
mean_latency_ms = round(float(_times_arr.mean()), 4)
p99_latency_ms = round(float(np.percentile(_times_arr, 99)), 4)
print(f' Mean latency : {mean_latency_ms:.4f} ms/sample')
print(f' P99 latency : {p99_latency_ms:.4f} ms/sample')
print('Done.')

```

CV sub-sample : 2,000 / 127,998

Fitting 5 folds for each of 5 candidates, totalling 25 fits

Best C : 0.001

Best CV F1 : 0.5880

Fitting final model on full training set...

Mean latency : 0.0146 ms/sample

P99 latency : 0.0268 ms/sample

Done.

6. CV Grid Search Results

```

In [7]: cv_results = pd.DataFrame(gs.cv_results_)

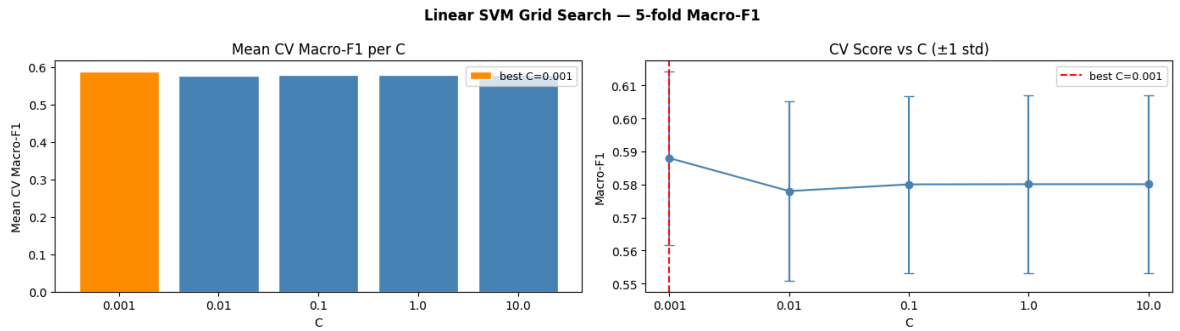
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
fig.suptitle('Linear SVM Grid Search - 5-fold Macro-F1',
             fontsize=12, fontweight='bold')

# Bar chart of mean CV score per C value
c_values = [str(c) for c in cv_results['param_svm__C']]
axes[0].bar(c_values, cv_results['mean_test_score'],
            color='steelblue', edgecolor='white')
axes[0].set_xlabel('C'); axes[0].set_ylabel('Mean CV Macro-F1')
axes[0].set_title('Mean CV Macro-F1 per C')
best_idx = cv_results['mean_test_score'].idxmax()
axes[0].bar(str(best_C), cv_results.loc[best_idx, 'mean_test_score'],
            color='darkorange', edgecolor='white', label=f'best C={best_C}')
axes[0].legend(fontsize=9)

# Line plot with error bars
axes[1].errorbar(
    c_values,
    cv_results['mean_test_score'],
    yerr=cv_results['std_test_score'],
    marker='o', color='steelblue', capsize=4
)
axes[1].axvline(str(best_C), color='red', ls='--', lw=1.5,
                label=f'best C={best_C}')
axes[1].set_xlabel('C'); axes[1].set_ylabel('Macro-F1')
axes[1].set_title('CV Score vs C ( $\pm 1$  std)')
axes[1].legend(fontsize=9)

plt.tight_layout(); plt.show()

```



7. Per-Class Metrics Table

```
In [8]: report = classification_report(
        y_test, y_pred, target_names=le.classes_, output_dict=True
    )
    report_df = pd.DataFrame(report).T
    report_df = report_df.drop(index=['accuracy'], errors='ignore')
    report_df['support'] = report_df['support'].astype(int)

    def highlight_metrics(col):
        per_class = list(le.classes_)
        if col.name not in {'precision', 'recall', 'f1-score'}:
            return [''] * len(col)
        vals = col.loc[per_class]
        styles = [''] * len(col)
        styles[report_df.index.get_loc(vals.idxmax())] = 'font-weight:bold; color:#1b7842'
        styles[report_df.index.get_loc(vals.idxmin())] = 'font-weight:bold; color:#d62728'
        return styles

    report_df.style \
        .apply(highlight_metrics) \
        .format({'precision': '{:.4f}', 'recall': '{:.4f}', 'f1-score': '{:.4f}'}) \
        .set_caption('Per-class metrics — green = best, red = worst among the 8 classes')
```

Out[8]: Per-class metrics — green = best, red = worst among the 8 classes

	precision	recall	f1-score	support
bamital	0.9976	1.0000	0.9988	417
cryptolocker	0.8048	0.9644	0.8774	7451
gameoverdga	0.4803	0.4391	0.4588	1692
goz	0.9692	0.9234	0.9457	1227
legit	0.9827	0.9926	0.9876	16000
necurs	0.1304	0.0072	0.0137	1666
newgoz	0.5276	0.5560	0.5414	1856
nivdort	0.9780	0.9994	0.9886	1691
macro avg	0.7338	0.7353	0.7265	32000
weighted avg	0.8434	0.8779	0.8560	32000

8. Confusion Matrix

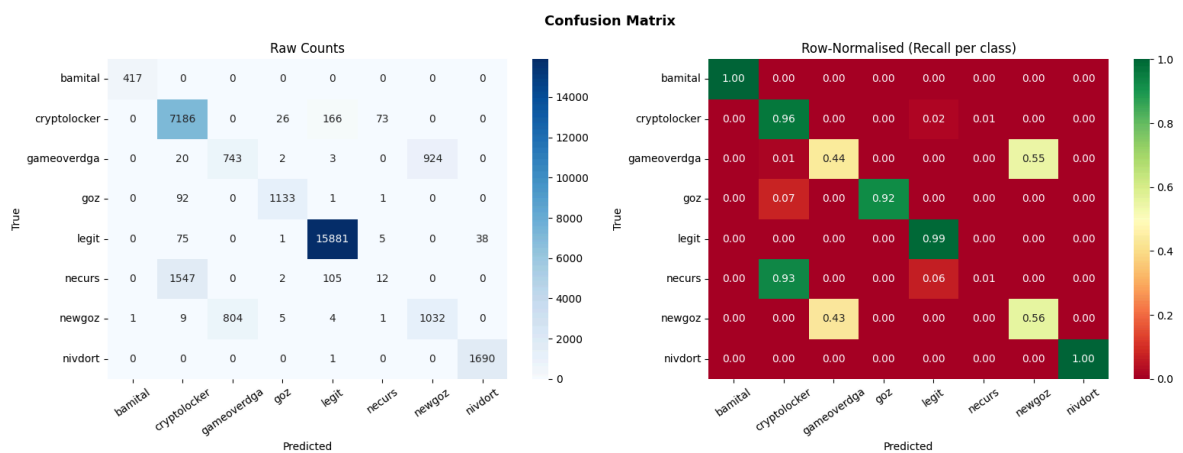
```
In [9]: cm = confusion_matrix(y_test, y_pred)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Confusion Matrix', fontsize=13, fontweight='bold')

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
             xticklabels=le.classes_, yticklabels=le.classes_, ax=axes[0])
axes[0].set_title('Raw Counts')
axes[0].set_xlabel('Predicted'); axes[0].set_ylabel('True')
axes[0].tick_params(axis='x', rotation=35)

cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)
sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='RdYlGn', vmin=0, vmax=1,
            xticklabels=le.classes_, yticklabels=le.classes_, ax=axes[1])
axes[1].set_title('Row-Normalised (Recall per class)')
axes[1].set_xlabel('Predicted'); axes[1].set_ylabel('True')
axes[1].tick_params(axis='x', rotation=35)

plt.tight_layout(); plt.show()
```



9. t-SNE Embedding

PCA-50 is applied first to speed up t-SNE and reduce noise. A balanced per-class sub-sample is used for tractability.

```
In [10]: EMBED_SAMPLES = 8000

idx_embed = np.concatenate([
    np.random.choice(
        np.where(y == c)[0],
        min(EMBED_SAMPLES // N_CLASSES, (y == c).sum()),
        replace=False
    ) for c in range(N_CLASSES)
])
X_embed = X_dense[idx_embed]
y_embed = y[idx_embed]

X_embed_s = StandardScaler().fit_transform(X_embed)
pca50 = PCA(n_components=50, random_state=SEED)
X_pca50 = pca50.fit_transform(X_embed_s)
```

```

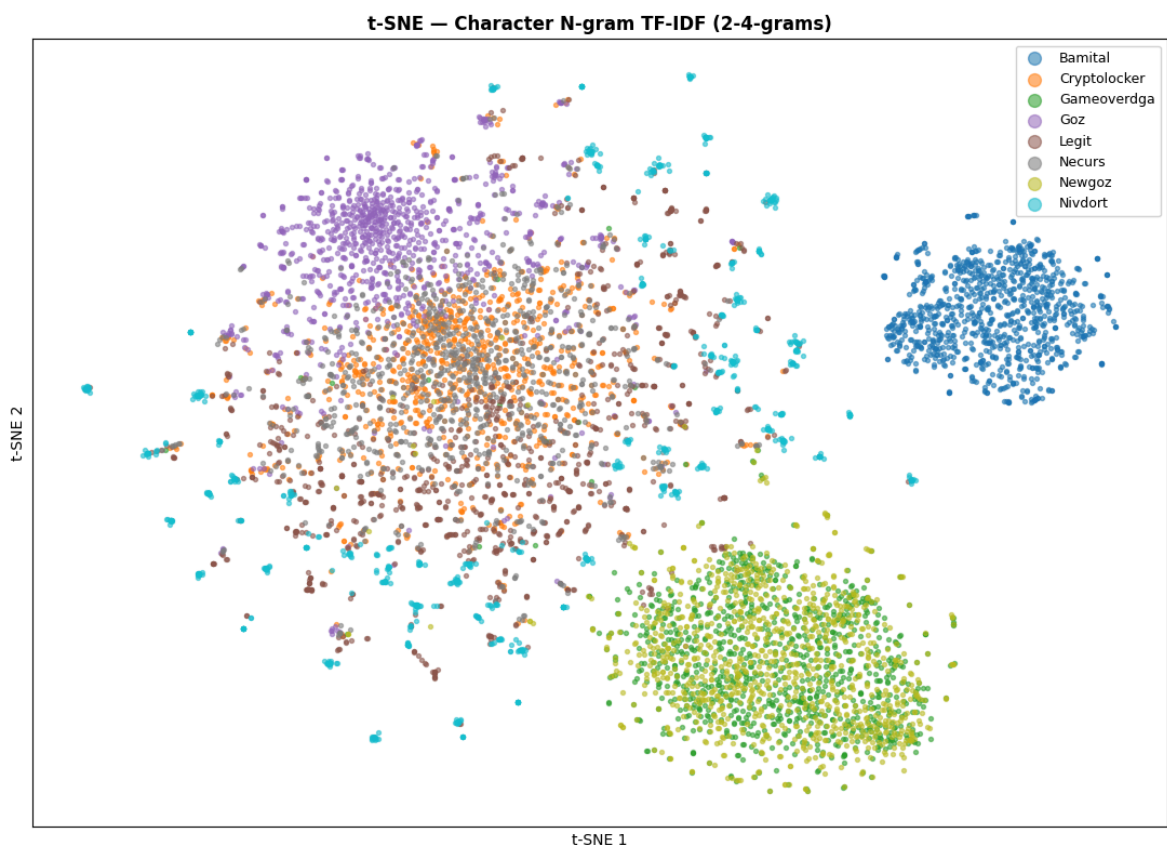
print(f'PCA-50 explains {pca50.explained_variance_ratio_.sum():.1%} of variance')

print(f'Running t-SNE on {len(idx_embed):,} samples...')
X_tsne = TSNE(n_components=2, perplexity=40,
              random_state=SEED, n_jobs=-1).fit_transform(X_pca50)
print('t-SNE done.')

fig, ax = plt.subplots(figsize=(11, 8))
for c, name in enumerate(le.classes_):
    m = y_embed == c
    ax.scatter(X_tsne[m, 0], X_tsne[m, 1], c=[CLASS_COLORS[c]],
              s=8, alpha=0.55, label=name.capitalize())
ax.set_title('t-SNE - Character N-gram TF-IDF (2-4-grams)',
            fontsize=12, fontweight='bold')
ax.set_xlabel('t-SNE 1'); ax.set_ylabel('t-SNE 2')
ax.legend(markerscale=3, fontsize=9)
ax.set_xticks([]); ax.set_yticks([])
plt.tight_layout(); plt.show()

```

PCA-50 explains 7.5% of variance
Running t-SNE on 8,000 samples...
t-SNE done.



10. UMAP Embedding

```

In [11]: print(f'Running UMAP on {len(idx_embed):,} samples...')
X_umap = umap.UMAP(n_components=2, n_neighbors=30, min_dist=0.1,
                  metric='euclidean', random_state=SEED,
                  low_memory=False).fit_transform(X_pca50)
print('UMAP done.')

fig, ax = plt.subplots(figsize=(11, 8))
for c, name in enumerate(le.classes_):
    m = y_embed == c

```

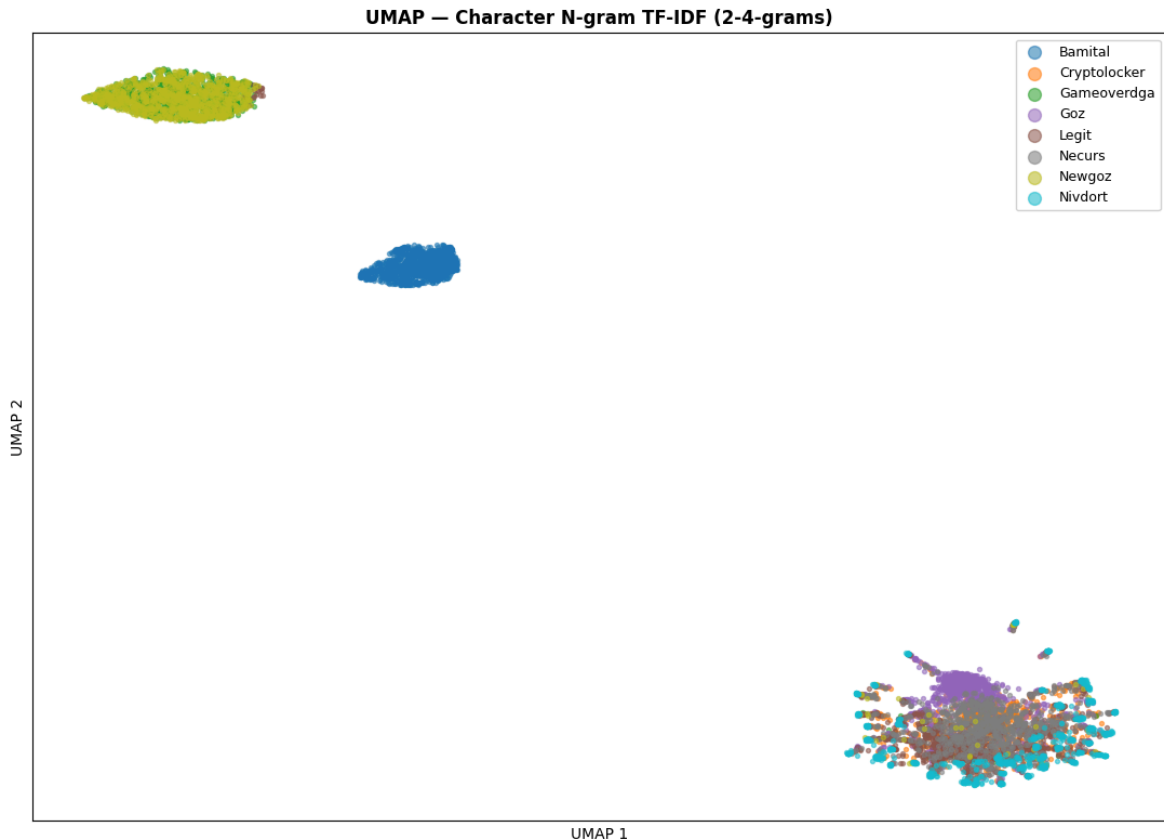
```

ax.scatter(X_umap[m, 0], X_umap[m, 1], c=[CLASS_COLORS[c]],
           s=8, alpha=0.55, label=name.capitalize())
ax.set_title('UMAP - Character N-gram TF-IDF (2-4-grams)',
             fontsize=12, fontweight='bold')
ax.set_xlabel('UMAP 1'); ax.set_ylabel('UMAP 2')
ax.legend(markerscale=3, fontsize=9)
ax.set_xticks([]); ax.set_yticks([])
plt.tight_layout(); plt.show()

```

Running UMAP on 8,000 samples...

UMAP done.



11. t-SNE vs UMAP Side-by-Side

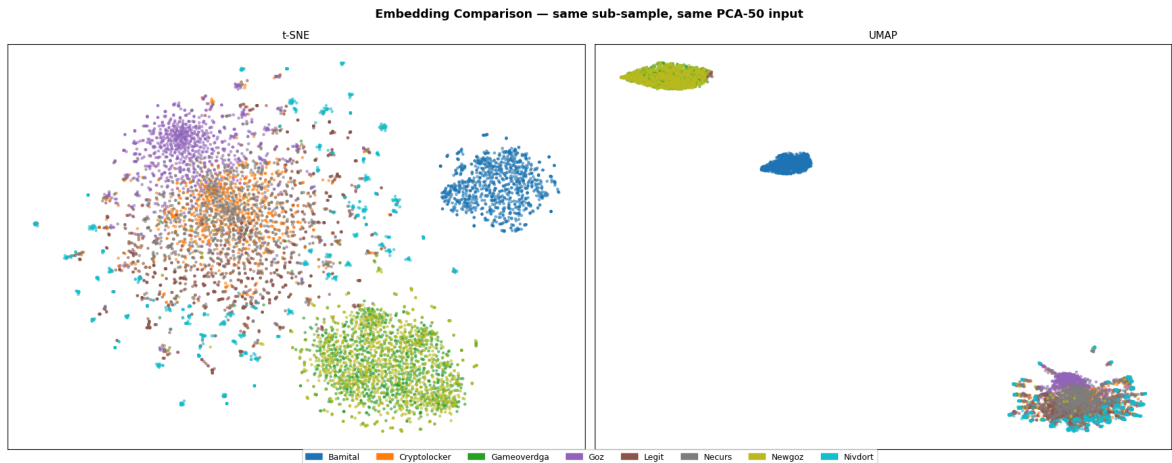
```

In [12]: fig, axes = plt.subplots(1, 2, figsize=(18, 7))
fig.suptitle('Embedding Comparison - same sub-sample, same PCA-50 input',
             fontsize=13, fontweight='bold')

for ax, X_2d, title in zip(axes, [X_tsne, X_umap], ['t-SNE', 'UMAP']):
    for c, name in enumerate(le.classes_):
        m = y_embed == c
        ax.scatter(X_2d[m, 0], X_2d[m, 1], c=[CLASS_COLORS[c]],
                  s=6, alpha=0.5)
    ax.set_title(title, fontsize=11)
    ax.set_xticks([]); ax.set_yticks([])

handles = [mpatches.Patch(color=CLASS_COLORS[i],
                           label=le.classes_[i].capitalize()) for i in range(N_CLASSES)]
fig.legend(handles=handles, loc='lower center', ncol=N_CLASSES,
           fontsize=9, bbox_to_anchor=(0.5, -0.02))
plt.tight_layout(); plt.show()

```



12. Centroid Distance Matrix

Class centroids are computed in the standardised TF-IDF space. Close centroids predict off-diagonal mass in the confusion matrix.

```
In [13]: X_dense_s = StandardScaler().fit_transform(X_dense)
centroids = np.array([X_dense_s[y == c].mean(axis=0) for c in range(N_CLASSES)])
dist_matrix = cdist(centroids, centroids, metric='euclidean')
cls_labels = [c.capitalize() for c in le.classes_]
dist_df = pd.DataFrame(dist_matrix, index=cls_labels, columns=cls_labels)

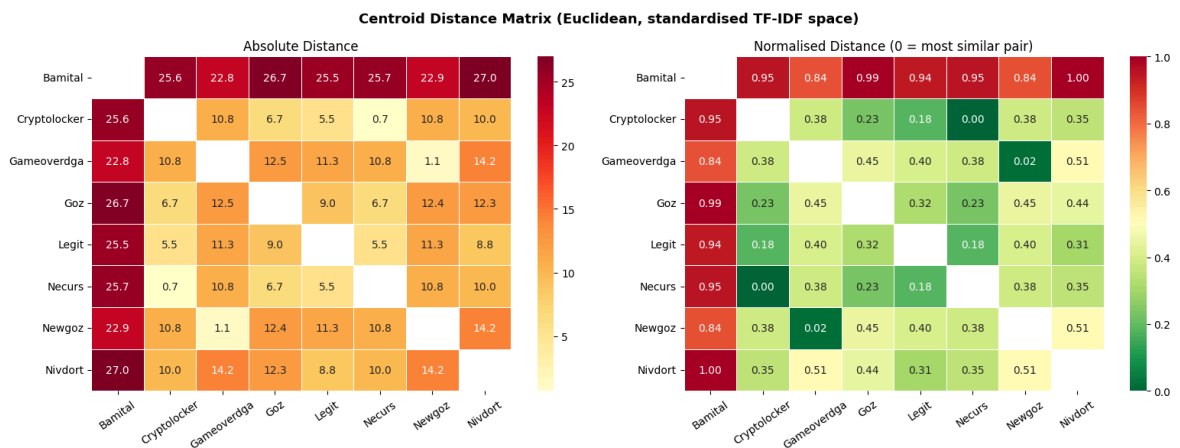
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Centroid Distance Matrix (Euclidean, standardised TF-IDF space)',
             fontsize=13, fontweight='bold')

mask_diag = np.eye(N_CLASSES, dtype=bool)
sns.heatmap(dist_df, annot=True, fmt='.1f', cmap='YlOrRd',
            mask=mask_diag, ax=axes[0], linewidths=0.5)
axes[0].set_title('Absolute Distance')
axes[0].tick_params(axis='x', rotation=35)

flat = dist_matrix[~mask_diag]
dist_norm = (dist_matrix - flat.min()) / (flat.max() - flat.min())
np.fill_diagonal(dist_norm, np.nan)
sns.heatmap(pd.DataFrame(dist_norm, index=cls_labels, columns=cls_labels),
            annot=True, fmt='.2f', cmap='RdYlGn_r',
            ax=axes[1], linewidths=0.5, vmin=0, vmax=1)
axes[1].set_title('Normalised Distance (0 = most similar pair)')
axes[1].tick_params(axis='x', rotation=35)

plt.tight_layout(); plt.show()

pairs = [(dist_matrix[i, j], le.classes_[i], le.classes_[j])
         for i in range(N_CLASSES) for j in range(i+1, N_CLASSES)]
print('Most similar pairs:')
for d, a, b in sorted(pairs)[:3]:
    print(f' {a:15s} <-> {b:15s} dist={d:.2f}')
print('Most distinct pairs:')
for d, a, b in sorted(pairs, reverse=True)[:3]:
    print(f' {a:15s} <-> {b:15s} dist={d:.2f}')
```



Most similar pairs:

```
cryptolocker <-> necurs      dist=0.72
gameoverdga <-> newgoz      dist=1.14
cryptolocker <-> legit      dist=5.48
```

Most distinct pairs:

```
bamital <-> nivdort      dist=26.96
bamital <-> goz          dist=26.72
bamital <-> necurs       dist=25.65
```

13. Decision Boundary (2-D PCA Projection)

Linear SVM retrained on PCA-2 data using the best C from CV.

Visualisation aid only — accuracy will be lower than the full model.

```
In [14]: pca2 = PCA(n_components=2, random_state=SEED)
X_2d_all = pca2.fit_transform(StandardScaler().fit_transform(X_dense))
X_2d_tr, X_2d_te, y_2d_tr, y_2d_te = train_test_split(
    X_2d_all, y, test_size=0.2, random_state=SEED, stratify=y
)

from sklearn.calibration import CalibratedClassifierCV
svm_2d = CalibratedClassifierCV(
    LinearSVC(C=best_C, class_weight='balanced', max_iter=2000, random_state=SEED,
    cv=3
)
svm_2d.fit(X_2d_tr, y_2d_tr)
acc_2d = (svm_2d.predict(X_2d_te) == y_2d_te).mean()
print(f'2-D accuracy: {acc_2d:.3f} '
      f'(PCA-2 = {pca2.explained_variance_ratio_.sum():.1%} variance)')

h = (X_2d_all.ptp(axis=0).mean()) / 300
pad = 0.5
xx, yy = np.meshgrid(
    np.arange(X_2d_all[:, 0].min()-pad, X_2d_all[:, 0].max()+pad, h),
    np.arange(X_2d_all[:, 1].min()-pad, X_2d_all[:, 1].max()+pad, h),
)
Z = svm_2d.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

fig, ax = plt.subplots(figsize=(11, 8))
ax.contourf(xx, yy, Z, alpha=0.22, cmap=plt.cm.get_cmap('tab10', N_CLASSES))
ax.contour(xx, yy, Z, linewidths=0.6, colors='k', alpha=0.25)
for c, name in enumerate(1e.classes_):
    m = y_2d_te == c
    ax.scatter(X_2d_te[m, 0], X_2d_te[m, 1], c=CLASS_COLORS[c],
```

```

        s=10, alpha=0.7, label=name.capitalize(), edgecolors='none')
ax.set_title(f'Decision Boundary - 2-D PCA '
            f'(acc={acc_2d:.3f}, C={best_C}, kernel=linear)',
            fontsize=11, fontweight='bold')
ax.set_xlabel(f'PC1 ({pca2.explained_variance_ratio_[0]:.1%} var)')
ax.set_ylabel(f'PC2 ({pca2.explained_variance_ratio_[1]:.1%} var)')
ax.legend(markerscale=3, fontsize=8)
plt.tight_layout(); plt.show()

```

2-D accuracy: 0.715 (PCA-2 = 0.5% variance)

```

-----
AttributeError                                Traceback (most recent call last)
Cell In[14], line 17
    13 acc_2d = (svm_2d.predict(X_2d_te) == y_2d_te).mean()
    14 print(f'2-D accuracy: {acc_2d:.3f} '
    15       f'(PCA-2 = {pca2.explained_variance_ratio_.sum():.1%} variance)')
--> 17 h = (X_2d_all.ptp(axis=0).mean()) / 300
    18 pad = 0.5
    19 xx, yy = np.meshgrid(
    20     np.arange(X_2d_all[:, 0].min()-pad, X_2d_all[:, 0].max()+pad, h),
    21     np.arange(X_2d_all[:, 1].min()-pad, X_2d_all[:, 1].max()+pad, h),
    22 )

```

AttributeError: 'numpy.ndarray' object has no attribute 'ptp'

14. Feature Importance — Top TF-IDF N-grams per Class

RBF-SVM has no direct weight vector, so two proxy views are shown:

1. **Mean TF-IDF** — average score for that n-gram within the class
2. **Discriminative contrast** — class mean minus the mean across all other classes, isolating patterns that are high for one family but low elsewhere

```

In [15]: TOP_N = 12
tfidf_names = np.array(tfidf.get_feature_names_out())

class_means = np.array([
    X_train_d[y_train == c].mean(axis=0) for c in range(N_CLASSES)
])
overall_mean = class_means.mean(axis=0)
contrast_scores = class_means - (
    (overall_mean * N_CLASSES - class_means) / (N_CLASSES - 1)
)

n_cols = 4
n_rows = math.ceil(N_CLASSES / n_cols)

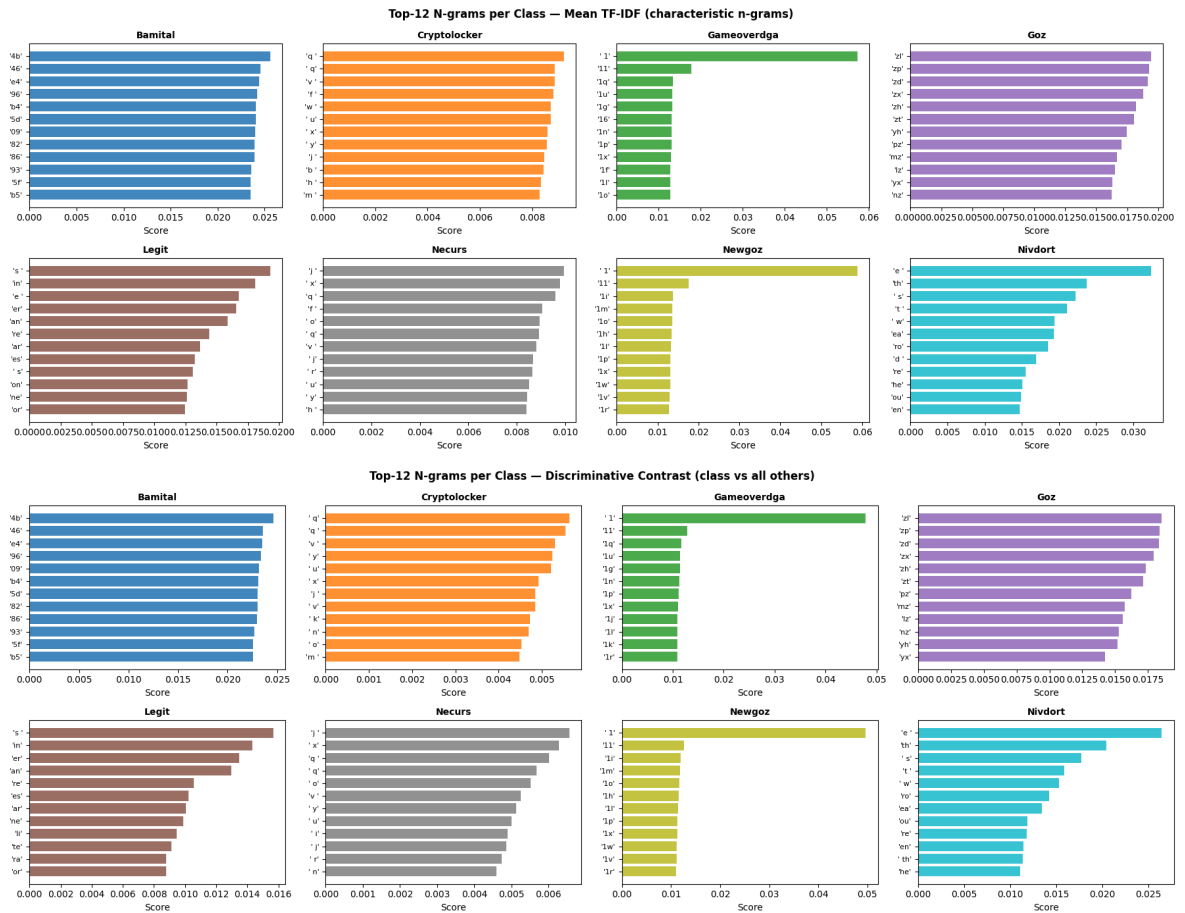
for scores_arr, subtitle in [
    (class_means, 'Mean TF-IDF (characteristic n-grams)'),
    (contrast_scores, 'Discriminative Contrast (class vs all others)'),
]:
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, n_rows * 3.5))
    fig.suptitle(f'Top-{TOP_N} N-grams per Class - {subtitle}',
                 fontsize=12, fontweight='bold')
    axes = axes.flatten()
    for c, name in enumerate(le.classes_):

```

```

scores = scores_arr[c]
top_idx = np.argsort(scores)[-TOP_N][::-1]
ax = axes[c]
ax.barh(range(TOP_N), scores[top_idx][::-1],
        color=CLASS_COLORS[c], alpha=0.85)
ax.set_yticks(range(TOP_N))
ax.set_yticklabels(
    [f'"{tfidf_names[i]}"' for i in top_idx[::-1]], fontsize=8
)
ax.set_title(name.capitalize(), fontsize=10, fontweight='bold')
ax.set_xlabel('Score')
for j in range(N_CLASSES, len(axes)):
    axes[j].set_visible(False)
plt.tight_layout(); plt.show()

```



15. Summary

```

In [16]: macro_f1 = f1_score(y_test, y_pred, average='macro')
micro_f1 = f1_score(y_test, y_pred, average='micro')
weighted_f1 = f1_score(y_test, y_pred, average='weighted')
accuracy = (y_pred == y_test).mean()
per_class_f1 = f1_score(y_test, y_pred, average=None)

easiest = le.classes_[per_class_f1.argmax()]
hardest = le.classes_[per_class_f1.argmin()]
closest = min([(dist_matrix[i,j], le.classes_[i], le.classes_[j])
               for i in range(N_CLASSES) for j in range(i+1, N_CLASSES)])

print('=' * 60)
print(' LINEAR SVM MULTI-CLASS DGA - SUMMARY')
print('=' * 60)

```

```

print(f' Kernel      : Linear (C={best_C})')
print(f' Features    : {X_dense.shape[1]:,} TF-IDF n-grams (char_wb, 2-4)')
print(f' Test accuracy : {accuracy:.4f}')
print(f' Macro F1      : {macro_f1:.4f}')
print(f' Micro F1       : {micro_f1:.4f}')
print(f' Weighted F1    : {weighted_f1:.4f}')
print()
print(f' Easiest class  : {easiest} (F1={per_class_f1.max():.4f})')
print(f' Hardest class   : {hardest} (F1={per_class_f1.min():.4f})')
print()
print(f' Closest centroid pair : {closest[1]} <-> {closest[2]}')
print(f'   dist={closest[0]:.2f} -- most likely source of confusion')
print(f' Mean latency    : {mean_latency_ms:.4f} ms/sample')
print(f' P99 latency     : {p99_latency_ms:.4f} ms/sample')
print('=' * 60)
print('\n Per-class F1:')
for cls, score in zip(le.classes_, per_class_f1):
    bar = chr(9608) * int(score * 30)
    print(f' {cls:15s} {bar} {score:.4f}')

```

=====

LINEAR SVM MULTI-CLASS DGA – SUMMARY

=====

```

Kernel      : Linear (C=0.001)
Features     : 5,000 TF-IDF n-grams (char_wb, 2-4)
Test accuracy : 0.8779
Macro F1     : 0.7265
Micro F1     : 0.8779
Weighted F1  : 0.8560

```

```

Easiest class : bamital (F1=0.9988)
Hardest class : necurs (F1=0.0137)

```

```

Closest centroid pair : cryptolocker <-> necurs
  dist=0.72 -- most likely source of confusion
Mean latency  : 0.0146 ms/sample
P99 latency   : 0.0268 ms/sample

```

=====

Per-class F1:

bamital		0.9988
cryptolocker		0.8774
gameoverdga		0.4588
goz		0.9457
legit		0.9876
necurs		0.0137
newgoz		0.5414
nivdort		0.9886

16. Leave-One-Family-Out (LOFO) Evaluation

For each DGA family in turn, we:

1. **Train** on all data **excluding** that family (held-out family is invisible at train time).
2. **Test** on the held-out family (can the model detect/classify a family it has never seen?).

Two metrics per fold:

- **Binary detection rate** — fraction of held-out family domains predicted as *any* DGA (not `legit`).
- **Exact-class F1** — F1 for the specific held-out class label.

Legit class is always included in training/testing to keep the benign reference intact.

```
In [17]: from sklearn.preprocessing import LabelEncoder
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import f1_score, classification_report

DGA_FAMILIES = [c for c in CLASSES if c != 'legit']
lofo_results = []

print(f'Running LOFO across {len(DGA_FAMILIES)} families...\n')

for held_out in DGA_FAMILIES:
    train_mask = dga_dataset['subclass'] != held_out
    test_mask = dga_dataset['subclass'] == held_out

    df_train = dga_dataset[train_mask].copy()
    df_test = dga_dataset[test_mask].copy()

    tfidf_lofo = TfidfVectorizer(
        analyzer='char_wb', ngram_range=(2, 4),
        max_features=5000, sublinear_tf=True,
    )
    X_tr_lofo = tfidf_lofo.fit_transform(df_train['domain'].tolist())
    X_te_lofo = tfidf_lofo.transform(df_test['domain'].tolist())

    le_lofo = LabelEncoder()
    y_tr_lofo = le_lofo.fit_transform(df_train['subclass'].tolist())

    pipe_lofo = Pipeline([
        ('scaler', StandardScaler(with_mean=False)),
        ('svm', CalibratedClassifierCV(
            LinearSVC(C=best_C, class_weight='balanced',
                      max_iter=2000, random_state=SEED),
            cv=3
        ))
    ])
    pipe_lofo.fit(X_tr_lofo, y_tr_lofo)

    # Per-sample latency for this fold
    _fold_times = []
    for _start in range(0, X_te_lofo.shape[0], BATCH_SIZE_LAT):
        _batch = X_te_lofo[_start : _start + BATCH_SIZE_LAT]
        _t0 = time.perf_counter()
        pipe_lofo.predict(_batch)
        _t1 = time.perf_counter()
        _ms = (_t1 - _t0) * 1000 / _batch.shape[0]
        _fold_times.extend([_ms] * _batch.shape[0])
    _fold_arr = np.array(_fold_times)
    fold_mean_lat = round(float(_fold_arr.mean()), 4)
    fold_p99_lat = round(float(np.percentile(_fold_arr, 99)), 4)

    y_pred_lofo = pipe_lofo.predict(X_te_lofo)
```

```

pred_names = le_lofo.inverse_transform(y_pred_lofo)

detection_rate = (pred_names != 'legit').mean()

from collections import Counter
top_pred = Counter(pred_names).most_common(1)[0]

lofo_results.append({
    'held_out_family' : held_out,
    'n_test'          : len(df_test),
    'detection_rate'   : round(detection_rate, 4),
    'top_predicted_as' : top_pred[0],
    'top_pred_count'   : top_pred[1],
    'top_pred_pct'     : round(top_pred[1] / len(df_test), 4),
    'mean_latency_ms'  : fold_mean_lat,
    'p99_latency_ms'   : fold_p99_lat,
})
print(f' [{held_out:15s}] n={len(df_test):5,} '
      f'detection={detection_rate:.2%} '
      f'mostly_predicted_as={top_pred[0]} ({top_pred[1]/len(df_test):.1%})')

lofo_df = pd.DataFrame(lofo_results).set_index('held_out_family')
print('\nLOFO Summary Table:')
print(lofo_df.to_string())

```

Running LOFO across 7 families...

[bamital	]	n=2,086	detection=98.66%	mostly_predicted_as=newgoz (55.9%)
[cryptolocker	]	n=37,254	detection=92.66%	mostly_predicted_as=necurs (89.7%)
[gameoverdga	]	n=8,461	detection=99.99%	mostly_predicted_as=newgoz (99.5%)
[goz	]	n=6,136	detection=99.64%	mostly_predicted_as=cryptolocker (92.3%)
[necurs	]	n=8,331	detection=93.84%	mostly_predicted_as=cryptolocker (93.4%)
[newgoz	]	n=9,276	detection=99.92%	mostly_predicted_as=gameoverdga (99.2%)
[nivdort	]	n=8,456	detection=0.04%	mostly_predicted_as=legit (100.0%)

LOFO Summary Table:

	n_test	detection_rate	top_predicted_as	top_pred_count	top_pre d_pct	mean_latency_ms	p99_latency_ms
held_out_family							
bamital	2086	0.9866	newgoz	1166			
0.5590	0.0189	0.0506					
cryptolocker	37254	0.9266	necurs	33412			
0.8969	0.0132	0.0256					
gameoverdga	8461	0.9999	newgoz	8416			
0.9947	0.0179	0.0350					
goz	6136	0.9964	cryptolocker	5663			
0.9229	0.0153	0.0266					
necurs	8331	0.9384	cryptolocker	7777			
0.9335	0.0086	0.0150					
newgoz	9276	0.9992	gameoverdga	9201			
0.9919	0.0164	0.0256					
nivdort	8456	0.0004	legit	8453			
0.9996	0.0124	0.0226					

16b. LOFO Detection Rate — Bar Chart

```
In [18]: fig, ax = plt.subplots(figsize=(10, 5))

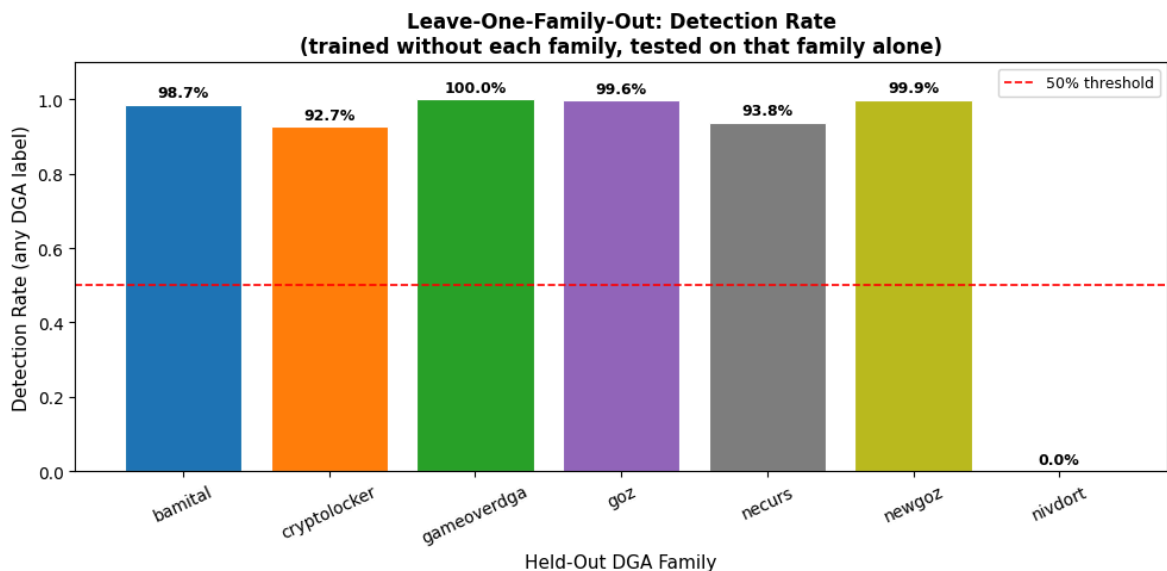
families = lofo_df.index.tolist()
rates     = lofo_df['detection_rate'].tolist()
colors    = [CLASS_COLORS[CLASSES.index(f)] for f in families]

bars = ax.bar(families, rates, color=colors, edgecolor='white', linewidth=0.8)

for bar, rate in zip(bars, rates):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01,
            f'{rate:.1%}', ha='center', va='bottom', fontsize=9, fontweight='bold')

ax.axhline(0.5, color='red', linestyle='--', linewidth=1.2, label='50% threshold')
ax.set_ylim(0, 1.1)
ax.set_xlabel('Held-Out DGA Family', fontsize=11)
ax.set_ylabel('Detection Rate (any DGA label)', fontsize=11)
ax.set_title('Leave-One-Family-Out: Detection Rate\n'
            '(trained without each family, tested on that family alone)',
            fontsize=12, fontweight='bold')
ax.tick_params(axis='x', rotation=25)
ax.legend(fontsize=9)
plt.tight_layout()
plt.show()

print('\nFamilies above 50% detection (model generalises to unseen family):')
for f, r in zip(families, rates):
    marker = '✓' if r >= 0.5 else 'X'
    print(f' {marker} {f:15s}: {r:.1%}')
```



Families above 50% detection (model generalises to unseen family):

```
✓ bamital      : 98.7%
✓ cryptolocker : 92.7%
✓ gameoverdga  : 100.0%
✓ goz          : 99.6%
✓ necurs       : 93.8%
✓ newgoz       : 99.9%
X nivdort      : 0.0%
```

17. Latency Analysis

Measures **per-sample inference latency** at prediction time:

- **Mean latency (ms)** — average wall-clock time to classify one domain.
- **P99 latency (ms)** — 99th-percentile tail latency; reflects worst-case production cost.

Latency is measured by running `predict()` in batches of 256 over the test set and dividing elapsed time by batch size. The LOFO section repeats this per fold so you can see whether training-set size affects inference speed.

```
In [19]: # — Main-model Latency summary —————
print('Main model latency (full test set)')
print(f' Mean : {mean_latency_ms:.4f} ms/sample')
print(f' P99  : {p99_latency_ms:.4f} ms/sample')
print()

# — LOFO per-fold latency table —————
lat_cols = ['mean_latency_ms', 'p99_latency_ms']
lat_df    = lofo_df[lat_cols].copy()

mean_row = lat_df.mean(numeric_only=True).rename('MEAN')
lat_df    = pd.concat([lat_df, mean_row.to_frame().T])

lat_df.style \
    .background_gradient(cmap='YlOrRd', subset=lat_cols) \
    .format(precision=4) \
    .set_caption('Per-sample inference latency by LOFO fold (ms) — lower is better')
```

Main model latency (full test set)

Mean : 0.0146 ms/sample
P99 : 0.0268 ms/sample

Out[19]: Per-sample inference latency by LOFO fold (ms) — lower is better

	mean_latency_ms	p99_latency_ms
bamital	0.0189	0.0506
cryptolocker	0.0132	0.0256
gameoverdga	0.0179	0.0350
goz	0.0153	0.0266
nekurs	0.0086	0.0150
newgoz	0.0164	0.0256
nivdort	0.0124	0.0226
MEAN	0.0147	0.0287

```
In [20]: # — Latency bar chart —————
fam_labels = lofo_df.index.tolist()
mean_lats  = lofo_df['mean_latency_ms'].tolist()
p99_lats   = lofo_df['p99_latency_ms'].tolist()
```

```

x      = np.arange(len(fam_labels))
width  = 0.35

fig, axes = plt.subplots(1, 2, figsize=(16, 5))
fig.suptitle('SVM Inference Latency – LOFO Folds', fontsize=13, fontweight='bold')

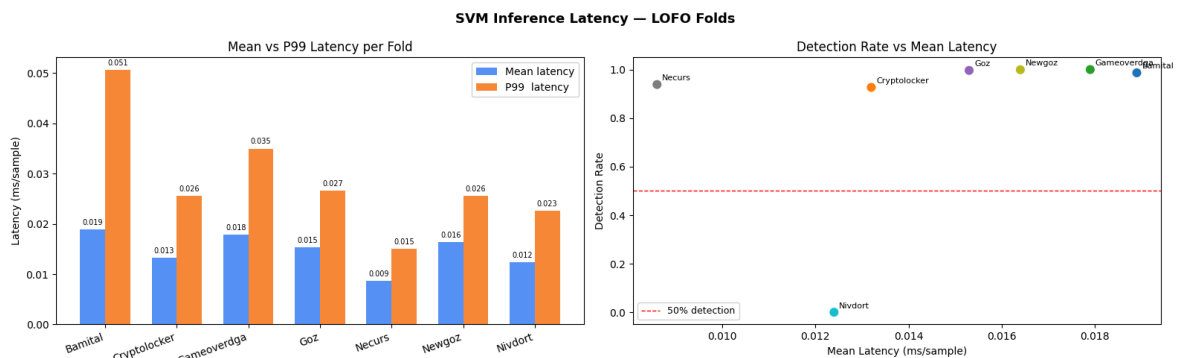
# Left: grouped bar – mean vs p99 per family
ax = axes[0]
b1 = ax.bar(x - width/2, mean_lats, width, label='Mean latency', color='#3b82f6')
b2 = ax.bar(x + width/2, p99_lats, width, label='P99 latency', color='#f97316')
ax.set_xticks(x)
ax.set_xticklabels([f.capitalize() for f in fam_labels], rotation=20, ha='right')
ax.set_ylabel('Latency (ms/sample)')
ax.set_title('Mean vs P99 Latency per Fold')
ax.bar_label(b1, fmt='%.3f', padding=3, fontsize=7)
ax.bar_label(b2, fmt='%.3f', padding=3, fontsize=7)
ax.legend()

# Right: scatter – detection rate vs mean latency
ax2 = axes[1]
det_rates = lofo_df['detection_rate'].tolist()
colors_sc = [CLASS_COLORS[CLASSES.index(f)] for f in fam_labels]
ax2.scatter(mean_lats, det_rates, c=colors_sc, s=90, zorder=3, edgecolors='white')
for fam, ml, dr in zip(fam_labels, mean_lats, det_rates):
    ax2.annotate(fam.capitalize(), (ml, dr), textcoords='offset points',
                 xytext=(5, 4), fontsize=8)
ax2.axhline(0.5, color='red', linestyle='--', linewidth=1, label='50% detection')
ax2.set_xlabel('Mean Latency (ms/sample)')
ax2.set_ylabel('Detection Rate')
ax2.set_title('Detection Rate vs Mean Latency')
ax2.legend(fontsize=9)

plt.tight_layout()
plt.show()

print(f'Overall mean latency (LOFO mean): {np.mean(mean_lats):.4f} ms/sample')
print(f'Overall P99 latency (LOFO mean): {np.mean(p99_lats):.4f} ms/sample')

```



Overall mean latency (LOFO mean): 0.0147 ms/sample

Overall P99 latency (LOFO mean): 0.0287 ms/sample

In []: